

POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Relazione Progettuale della Prova Finale di Reti Logiche

Laurea Triennale in
Ingegneria Informatica

Autore: **Pietro Ghersetich**

Anno Accademico: 2024-2025

Indice

Indice **i**

1 Introduzione **1**

2 Architettura del Sistema **3**
2.1 Modulo Principale: `project_reti_logiche` 4
2.2 Contatore: `counter` 4
2.3 Finestra Scorrevole: `sliding_window` 4
2.4 Unità Multiply-Accumulate: `mac_unit` 5
2.5 Unità di Normalizzazione: `normalization_unit` 5
2.6 Calcolatore di Indirizzi: `address_calculator` 5
2.7 Macchina a Stati Finiti (FSM): `fsm` 5

3 Risultati Sperimentali **8**
3.1 Sintesi 8
3.2 Simulazioni 8
3.2.1 Test 1: Lettura Parametri e Selezione Coefficienti 9
3.2.2 Test 2: Funzionamento con K Minimo (K=7), Filtro Ordine 3, Padding e Saturazione 10
3.2.3 Test 3: Sequenza Corta (K=3), Filtro Ordine 5 e Padding Intensivo con Valori Standard 10
3.2.4 Test 4: Gestione Esecuzioni Multiple e Reset del Sistema 11

4 Conclusioni **12**

1 | Introduzione

Il presente progetto è focalizzato sullo sviluppo di un modulo hardware, descritto in VHDL, capace di applicare un filtro differenziale su sequenze di dati numerici. Il sistema, come rappresentato in Figura 1.1, interagisce con una memoria esterna per leggere i dati di ingresso, inclusi i parametri di configurazione, e per scrivere i risultati dell'elaborazione.

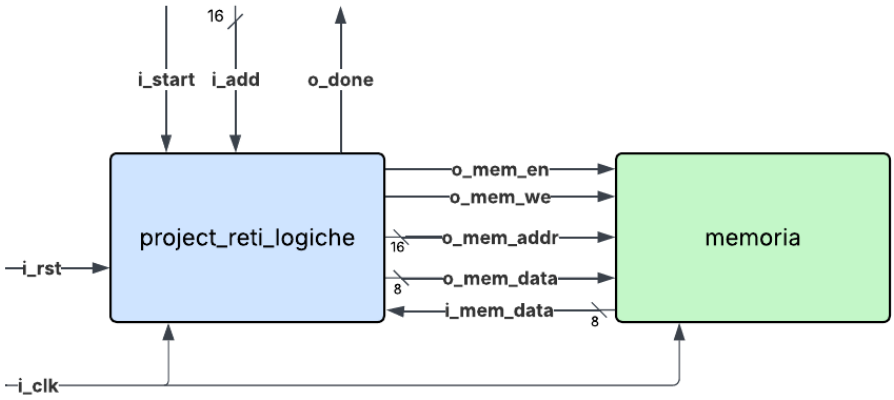


Figura 1.1: Schema a blocchi generale del sistema e interfaccia con la memoria.

L'operatività del modulo hardware inizia con la lettura di un'intestazione di 17 byte dalla memoria. Questa intestazione fornisce i parametri essenziali per l'elaborazione:

- La lunghezza K della sequenza di campioni di ingresso W_i, il cui ognuno di essi è una parola a 8 bit con segno (valori da -128 a +127).
- Il tipo di filtro da impiegare (ordine 3 oppure ordine 5), determinato dal contenuto del byte S.
- I 14 byte che contengono i coefficienti C_j necessari per entrambi gli ordini di filtro (sette coefficienti per ciascun ordine).

Il processo centrale consiste nell'applicare il filtro designato (ordine 3 o 5, in base al valore di S) a ogni campione W_i della sequenza. Per determinare il risultato filtrato R_i di un campione W_i, il sistema considera una finestra mobile di 7 campioni. Questa finestra è centrata sul campione W_i corrente e include i suoi campioni adiacenti. Nel caso in cui i campioni necessari per completare la finestra si trovino al di fuori dei limiti della sequenza W (cioè all'inizio o alla fine), si utilizzano valori nulli (padding con zeri) per le posizioni mancanti. Una volta composta la finestra di 7 campioni, si calcola un valore intermedio, V_i, prima della normalizzazione. Questo V_i è il risultato della somma dei prodotti tra i 7 coefficienti del filtro selezionato e i 7 campioni presenti nella finestra corrente.

Ad esempio, considerando il filtro di ordine 3 con coefficienti $C = [0, -1, 8, 0, -8, 1, 0]$ e una sequenza di input che inizia con $W_1 = 126, W_2 = -75, W_3 = 114, W_4 = 94, \ldots$, per calcolare il primo risultato R_1:

- La finestra corrente di 7 campioni, con padding, è $[0, 0, 0, 126, -75, 114, 94]$.
- Il valore non normalizzato $V_1 = (0 \cdot 0) + (-1 \cdot 0) + (8 \cdot 0) + (0 \cdot 126) + (-8 \cdot (-75)) + (1 \cdot 114) + (0 \cdot 94) = 714$.

Il valore V_i (un intero a 18 bit con segno) viene quindi normalizzato dividendolo per un fattore n ($n = 12$ per ordine 3, $n = 60$ per ordine 5). Tale divisione è approssimata, come da specifica, sommando i risultati di specifici shift aritmetici a destra di V_i. L'operazione $V_i \gg s_k$ indica uno shift aritmetico a destra di s_k posizioni del valore binario V_i. Ogni shift a destra di una posizione equivale a una divisione intera per due; pertanto, uno shift di s_k posizioni corrisponde a una divisione per 2^{s_k}. Nello shift aritmetico, il bit di segno di V_i viene preservato e replicato nelle posizioni più significative liberate dallo shift. Per $n = 12$, l'approssimazione $1/12 \approx 1/16 + 1/64 + 1/256 + 1/1024$ implica che $R_i \approx (V_i \gg 4) + (V_i \gg 6) + (V_i \gg 8) + (V_i \gg 10)$.

Se V_i è negativo, è però necessario, come da specifica, che a ciascun termine shiftato si aggiunga 1 prima della somma, per correggere il troncamento.

Nel nostro esempio, $V_1 = 714_{10} = 0000000001011001010_2$:

- $V_1 \gg 4$: 714_{10} shiftato a destra di 4 posizioni risulta in 44_{10}.
- $V_1 \gg 6$: 714_{10} shiftato a destra di 6 posizioni risulta in 11_{10}.
- $V_1 \gg 8$: 714_{10} shiftato a destra di 8 posizioni risulta in 2_{10}.
- $V_1 \gg 10$: 714_{10} shiftato a destra di 10 posizioni risulta in 0_{10}.
- Quindi, $R_1 = 44 + 11 + 2 + 0 = 57$.

Dopo la normalizzazione, i risultati vengono saturati nell'intervallo [-128, +127] e scritti in memoria a seguito delle K parole. L'intero processo è sincrono, governato da segnali di clock, start e reset, e termina con l'asserzione di un segnale di done. Nei capitoli successivi, verranno analizzate in dettaglio le scelte architetturali, il funzionamento dei singoli moduli implementati e la gestione di alcuni casi limite significativi.

2 | Architettura del Sistema

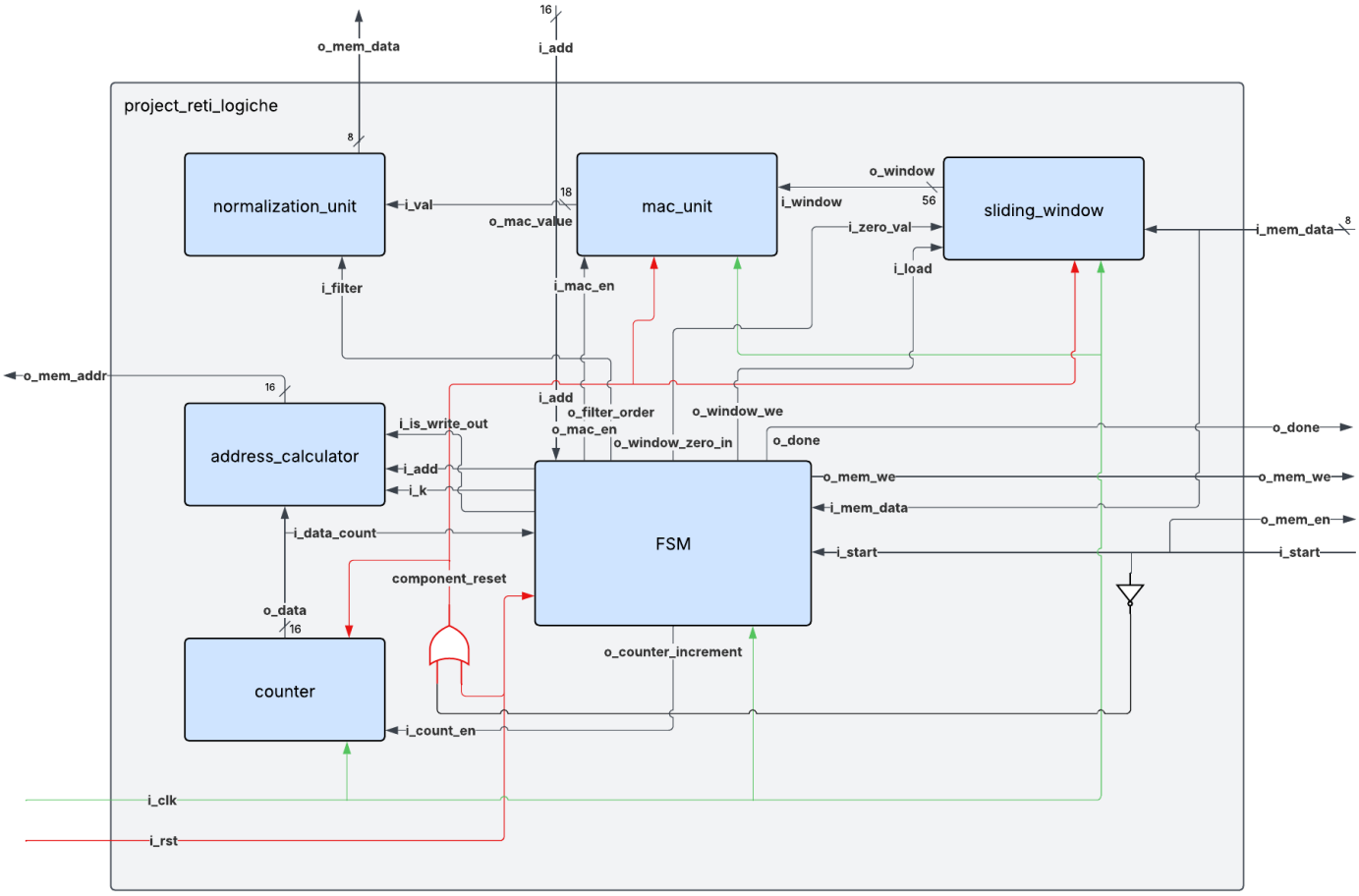


Figura 2.1: Rappresentazione dettagliata dell'architettura interna del modulo `project_reti_logiche`.

L'architettura del sistema è stata sviluppata seguendo un approccio modulare. Questa scelta mira a semplificare la progettazione, la verifica e la manutenibilità del codice VHDL. L'entità di vertice, `project_reti_logiche`, funge da contenitore per i vari sottomoduli e gestisce l'interfacciamento diretto con la memoria esterna e l'integrazione con i sottomoduli. Lo schema a blocchi completo dell'architettura interna è visibile in Figura 2.1. Gli aspetti generali dell'interazione tra i moduli e i segnali condivisi sono i seguenti:

- **Sincronizzazione**: Tutte le operazioni sequenziali dei componenti sono sincronizzate dal segnale di clock globale `i_clk`.
- **Reset del Sistema**: Il segnale di reset primario `i_rst`, di natura asincrona, inizializza l'intero sistema. In particolare, la `fsm` è resettata direttamente da `i_rst`.

- **Gestione del Reset dei Componenti**: Particolare attenzione è stata dedicata ai meccanismi di reset dei componenti sequenziali del percorso dati. Questi moduli (`sliding_window`, `counter` e `mac_unit`) sono resettati dal segnale interno `component_reset`. Tale segnale non dipende solo dal reset generale asincrono `i_rst`, ma anche dal segnale di avvio `i_start` basso, indicando che non è in corso alcuna computazione. Questa strategia permette ai suddetti moduli di ricominciare un ciclo di lavoro da uno stato noto senza dipendere da un intervento esplicito di `i_rst` ad ogni fine elaborazione.
- **Abilitazione della Memoria**: Il segnale `o_mem_en` è direttamente collegato a `i_start`. Ciò assicura che la memoria esterna sia sempre abilitata per le operazioni di lettura e scrittura quando il sistema è attivo.

2.1. Modulo Principale: `project_reti_logiche`

L'entità `project_reti_logiche`, definita dalle specifiche di progetto, è il modulo di vertice che integra i sottomoduli. Oltre a istanziarli e interconnetterli, gestisce direttamente segnali cruciali come l'abilitazione della memoria (`o_mem_en`), il segnale di reset per i componenti del datapath (`component_reset`) e l'inoltro dell'indirizzo di memoria calcolato (`o_mem_addr`).

2.2. Contatore: `counter`

Il `counter` è un semplice contatore a 16 bit. La sua funzione primaria è quella di tenere traccia del progresso generale dell'elaborazione, fornendo l'indice `i_data_count` alla FSM e al calcolatore di indirizzi. Questo indice rappresenta l'offset corrente rispetto all'indirizzo base della memoria. Si incrementa quando abilitato (tramite `i_count_en`) dalla FSM e viene resettato dal segnale `component_reset`.

2.3. Finestra Scorrevole: `sliding_window`

Questo modulo implementa una finestra scorrevole FIFO (First-In, First-Out) di 7 byte. Questo numero di byte è stato scelto in quanto l'unità MAC, che riceve i dati dalla finestra, opera sempre con un set di 7 coefficienti; questo numero corrisponde al massimo dei coefficienti, necessario per il filtro di ordine 5. Ad ogni ciclo di clock, se abilitata dal segnale di caricamento `i_load`, la finestra aggiorna il suo contenuto: i byte esistenti vengono shiftati e un nuovo byte viene caricato. Questo può essere un dato letto dalla memoria (`i_data`) o uno zero (controllato da `i_zero_val`).

Anche quando è selezionato il filtro di ordine 3, che utilizza meno termini, la struttura a 7 campioni e 7 coefficienti viene mantenuta a livello di interfaccia con l'unità MAC; la corretta gestione dei coefficienti attivi è demandata alla logica di controllo.

2.4. Unità Multiply-Accumulate: `mac_unit`

L'unità MAC è il cuore computazionale del filtro. Esegue la moltiplicazione e accumulo (sommatoria) dei 7 campioni a 8 bit presenti nella finestra scorrevole (`i_window`) per i corrispondenti 7 coefficienti del filtro (`i_coeffs`), anch'essi a 8 bit. Tutti i valori sono trattati come numeri con segno. Il risultato di questa operazione (`o_mac_value`) è un valore a 18 bit con segno. L'operazione MAC viene completata in un singolo ciclo di clock quando il modulo è abilitato dal segnale `i_mac_en`.

2.5. Unità di Normalizzazione: `normalization_unit`

Questo modulo combinatorio riceve il risultato a 18 bit dall'unità MAC e applica due trasformazioni finali: normalizzazione e saturazione. La normalizzazione scala il risultato del MAC dividendolo per il fattore n specifico del filtro (selezionato tramite `i_filter`), utilizzando l'approssimazione basata su shift e somma. Se il valore in ingresso al normalizzatore è negativo, viene sommato 1 a ciascun termine ottenuto dagli shift prima della loro somma finale, per ridurre l'effetto dell'errore causato dal troncamento verso il basso. Successivamente, il valore normalizzato viene saturato ai limiti [-128, +127].

L'uscita finale `o_filtered_value` è un valore a 8 bit con segno.

2.6. Calcolatore di Indirizzi: `address_calculator`

Questo blocco puramente combinatorio ha il compito di generare l'indirizzo di memoria effettivo da inviare alla memoria. Riceve in input l'indirizzo base (`i_add`), il valore corrente del contatore (`i_data_count`), la lunghezza della sequenza K, e il segnale di controllo `i_is_write_out` dalla FSM. Durante le fasi di lettura dei parametri e dei dati W_i, `i_is_write_out` è '0' e l'indirizzo generato è semplicemente `i_add + i_data_count`. Durante la fase di scrittura dei risultati R_i, la FSM imposta `i_is_write_out` a '1'. In questo scenario, il valore di `i_data_count` (che la FSM continua ad incrementare) viene utilizzato per determinare l'indice del campione W_i per il quale si è appena calcolato il risultato R_i. Per ottenere l'indice corretto del risultato R_i da scrivere (da 0 a K-1), si sottrae un offset dal valore di `i_data_count`. Questo offset tiene conto dei 17 byte dell'header e dei 4 cicli di inizializzazione della finestra, portando a un totale di 21. L'indirizzo di scrittura finale per R_i sarà quindi `i_add + 17 + i_k + (i_data_count - 21)`, garantendo che i risultati siano scritti sequenzialmente dopo il blocco dei dati di input W_i.

2.7. Macchina a Stati Finiti (FSM): `fsm`

La FSM è il cervello del sistema e ne governa l'intero comportamento. Attraverso una sequenza di stati, essa scandisce le fasi operative: lettura dei parametri iniziali (K, S), caricamento dei coefficienti del filtro, acquisizione dei dati dalla memoria, processamento tramite filtro, e infine scrittura dei risultati. Il diagramma di stato in Figura 2.2 illustra il flusso di controllo principale.

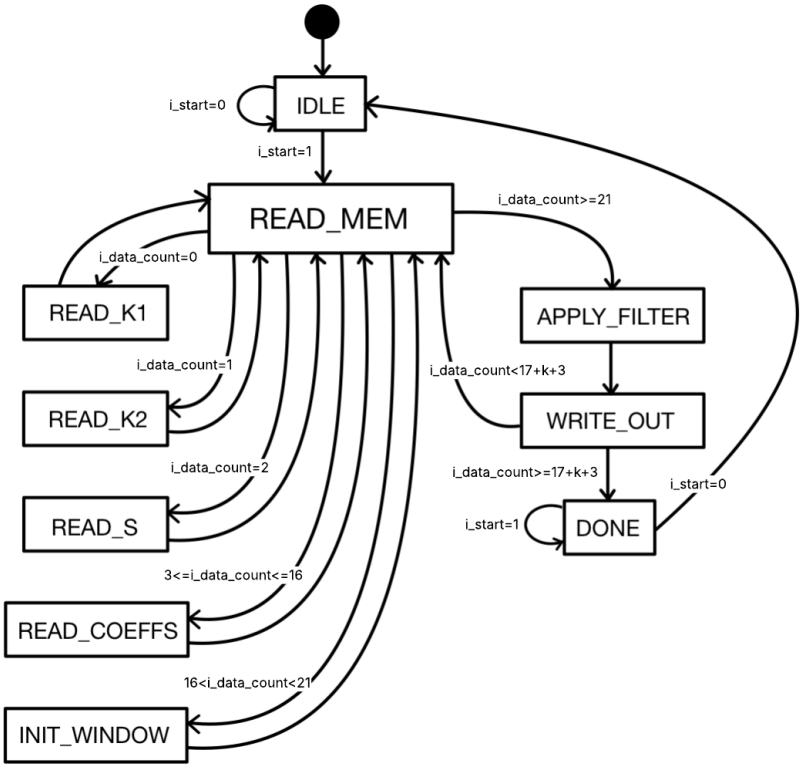


Figura 2.2: Diagramma degli stati della FSM.

Il diagramma mostra le transizioni tra gli stati principali basate sui segnali di input `i_start` e sul contatore `i_data_count`. Quest'ultimo, fornito dal modulo `counter`, rappresenta l'offset corrente rispetto all'indirizzo base della memoria e viene incrementato dalla FSM per scandire le varie fasi. Lo stato `READ_MEM` funge da hub centrale, indirizzando il flusso in base al valore del counter, oltre a permettere di "usare" un ciclo di clock per garantire al correttezza del segnale `i_mem_data`.

Oltre a orchestrare le transizioni di stato, la FSM è responsabile della generazione dei segnali di controllo per i vari moduli. Il suo funzionamento può essere suddiviso nelle seguenti fasi operative principali:

- **Fase di Configurazione Iniziale (`IDLE` → `READ_MEM` → `READ_K1/K2/S/COEFFS`)**: Dopo l'avvio, la FSM entra in una sequenza di stati per leggere i parametri fondamentali (K, S, 7 coefficienti) dalla memoria. Durante questi stati, la FSM abilita l'incremento del contatore (`o_counter_increment`) per leggere sequenzialmente i byte dalla memoria. Nello stato `READ_COEFFS`, in particolare, gestisce il caricamento selettivo di 7 dei 14 byte dei coefficienti, basandosi sul tipo di filtro `S`. Se è selezionato il filtro di ordine 3, la FSM assicura che i coefficienti C_0, C_6 siano caricati come zero nel registro, garantendo che l'unità MAC, pur operando sempre su 7 termini, utilizzi di fatto solo i coefficienti pertinenti, come da specifica.

- **Fase di Inizializzazione della Finestra (`READ_MEM` → `INIT_WINDOW`)**: Completata la lettura dell'header, la FSM avvia l'inizializzazione della finestra scorrevole. L'obiettivo è caricare i primi campioni della sequenza di input in modo che il primo dato effettivo (W1) si trovi al centro della finestra per il primo calcolo del filtro. Durante questi stati, la FSM attiva la scrittura nella finestra (`o_window_we`). Il segnale `o_window_zero_in` gestisce l'inserimento di un valore nullo: ciò avviene se, per la posizione corrente richiesta dalla finestra, non è disponibile un campione valido dalla sequenza di input (lunga K), assicurando così il padding con zeri. Questo è cruciale se la sequenza K è troppo corta per popolare adeguatamente la finestra con dati reali durante la sua inizializzazione.
- **Fase di Filtraggio e Scrittura (`READ_MEM` → `APPLY_FILTER` → `WRITE_OUT`)**: Questa è la fase operativa principale, ripetuta per ogni campione della sequenza. Durante lo stato `APPLY_FILTER` viene abilitata l'unità MAC, che calcola il prodotto e la somma dei campioni e dei coefficienti; questa operazione si completa in un singolo ciclo di clock. Il risultato a 18 bit generato dalla MAC è immediatamente processato, nello stesso ciclo di clock, dall'unità di Normalizzazione, la quale opera in modo puramente combinatorio. Parallelamente a queste operazioni di calcolo nello stato `APPLY_FILTER`, la finestra scorrevole viene aggiornata con il campione successivo (o uno zero di padding, se necessario), preparandosi per la prossima iterazione. Una volta che il risultato del filtro è stato calcolato e normalizzato, la FSM transita allo stato `WRITE_OUT`. Qui, il valore finale a 8 bit viene scritto nella memoria esterna nell'indirizzo fornito dall'`address_calculator` (attivando `o_mem_we` e `o_is_write_out` per il corretto indirizzamento di scrittura) e il contatore è incrementato per passare al campione seguente o per concludere l'elaborazione.
- **Fase di Completamento (`WRITE_OUT` → `DONE` → `IDLE`)**: Una volta elaborati e scritti tutti i K risultati (condizione verificata in `WRITE_OUT` confrontando `i_data_count` con K, tenendo conto degli offset iniziali e del numero di cicli per campione), la FSM transita nello stato `DONE`, alzando il segnale `o_done`. Rimane in questo stato finché `i_start` è alto, per poi tornare a `IDLE`.

3 | Risultati Sperimentali

Questa sezione è dedicata alla discussione dei risultati ottenuti dalla sintesi del progetto VHDL e delinea le verifiche funzionali attese tramite simulazione. L'obiettivo è dimostrare la corretta implementazione delle specifiche e il rispetto dei requisiti prestazionali.

3.1. Sintesi

La sintesi del progetto è stata effettuata utilizzando Xilinx Vivado v.2018.1, con target FPGA Artix-7 (specificamente il dispositivo 7a200tfbg484-1).

Utilizzo delle Risorse

Il report di utilization, presentato in Tabella 3.1, indica un impiego limitato delle risorse logiche dell'FPGA. In particolare, si nota l'assenza di Latch.

Tabella 3.1: Utilizzo Slice Logic (Sintesi).

Site Type	Used	Fixed	Available	Util%
Slice LUTs	742	0	134600	0.55
LUT as Logic	742	0	134600	0.55
LUT as Memory	0	0	46200	0.00
Slice Registers	178	0	269200	0.07
Register as Flip Flop	178	0	269200	0.07
Register as Latch	0	0	269200	0.00
F7 Muxes	0	0	67300	0.00
F8 Muxes	0	0	33650	0.00

Analisi Temporale

Il vincolo temporale principale per il progetto è un periodo di clock di 20 ns (frequenza di 50 MHz). Si noti che il timing report post-sintesi indica uno slack positivo (MET) di $+$**6.237ns**.

3.2. Simulazioni

Per validare la correttezza logica e funzionale del modulo `project_reti_logiche` implementato, sono state condotte diverse simulazioni. Oltre all'utilizzo di un testbench generale fornito, sono stati definiti alcuni casi di test specifici per esplorare il comportamento del sistema in scenari operativi chiave e in condizioni limite.

3.2.1. Test 1: Lettura Parametri e Selezione Coefficienti

Input:

- Parametri in memoria: K=7, S=LSB '0' (Filtro Ordine 3).
- Coefficienti per Ordine 3 in memoria: $[12, 103, -88, 18, -25, -22, -88]$.
- Coefficienti per Ordine 5 in memoria: $[123, 91, 13, -98, -91, 9, -41]$.
- Dati W_i: [...] (sequenza breve non critica per la verifica dei parametri).

Output Atteso (Verifica interna FSM e valori inviati alla MAC):

- Valore K registrato: 7.
- Selezione filtro: Ordine 3.
- Coefficienti inviati alla MAC: $[0, 103, -88, 18, -25, -22, 0]$.

Questo test preliminare è cruciale per assicurare che la FSM interpreti correttamente l'header dalla memoria, assembli K, identifichi l'ordine del filtro S e carichi i 7 coefficienti corretti. È inoltre verificato che, selezionando il filtro di ordine 3, i coefficienti C_0 e C_6 (il primo e l'ultimo dei sette coefficienti) inviati all'unità MAC siano effettivamente zero, per garantire al filtro di lavorare su 5 coefficienti, come da specifica, anche se nelle locazioni di memoria da cui ci si aspetterebbe di leggerli fossero presenti valori non nulli. Ciò è garantito dalla logica interna di caricamento della FSM.

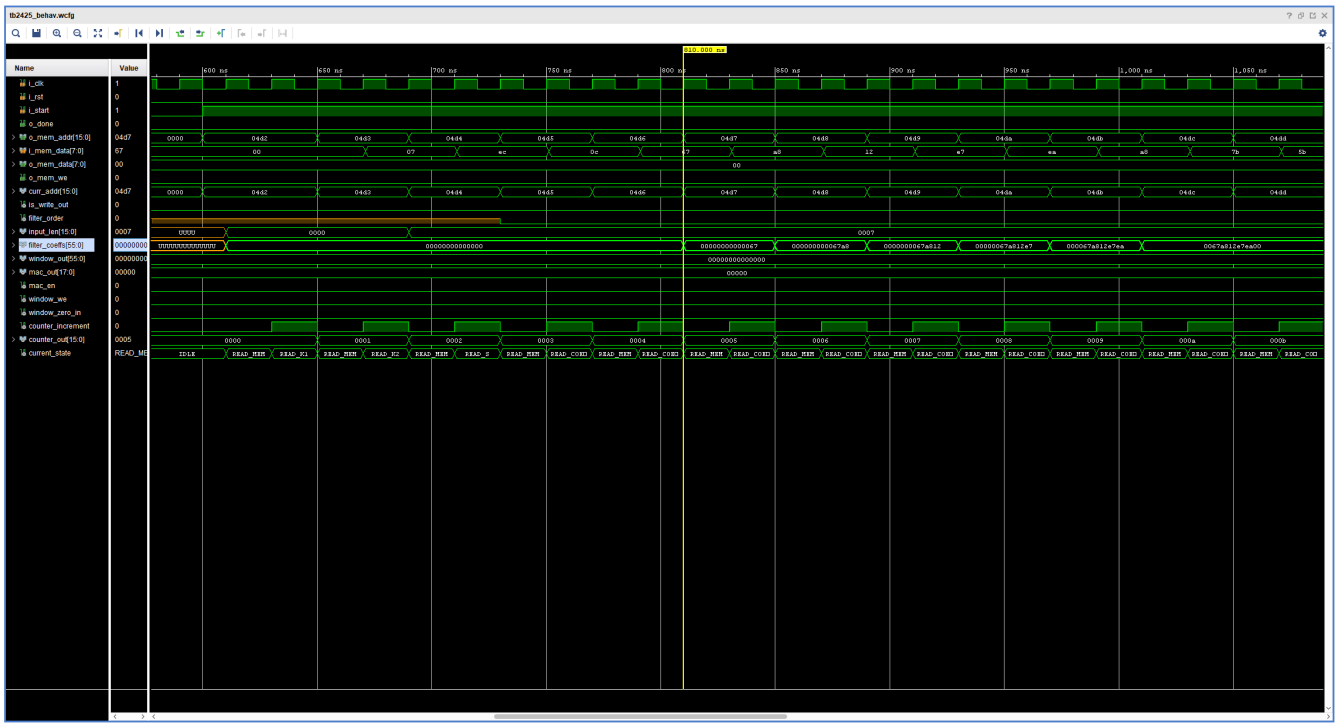


Figura 3.1: Waveform Test 1: Lettura Parametri.

3.2.2. Test 2: Funzionamento con K Minimo (K=7), Filtro Ordine 3, Padding e Saturazione

Input:

- Parametri in memoria: K=7, S=LSB '0' (Filtro Ordine 3).
- Coefficienti Ordine 3: $C_3 = [0, -1, 8, 0, -8, 1, 0]$.
- Dati W_i: $W = [127, 50, 0, -127, -50, 100, 110]$.

Output Atteso (Valori R_i in memoria e stati finestra):

- $W_1 = 127$:
  - Finestra Scorrevole: $[0, 0, 0, 127, 50, 0, -127] \longrightarrow R_1 = -31$.
- $W_2 = 50$:
  - Finestra Scorrevole: $[0, 0, 127, 50, 0, -127, -50] \longrightarrow R_2 = 71$.
- $W_3 = 0$:
  - Finestra Scorrevole: $[0, 127, 50, 0, -127, -50, 100] \longrightarrow R_3 = 101$.
- $W_4 = -127$:
  - Finestra Scorrevole: $[127, 50, 0, -127, -50, 100, 110] \longrightarrow R_4 = 36$.
- $W_5 = -50$:
  - Finestra Scorrevole: $[50, 0, -127, -50, 100, 110, 0]$
  - Valore Intermedio $V_5 = -1016 - 800 + 110 = -1706$
  - Normalizzazione ($n = 12$): Termini: $(-1706 \gg 4) + 1 = -107 + 1 = -106$; $(-1706 \gg 6) + 1 = -27 + 1 = -26$; $(-1706 \gg 8) + 1 = -7 + 1 = -6$; $(-1706 \gg 10) + 1 = -2 + 1 = -1$. Somma: $-106 - 26 - 6 - 1 = -139$.
  - Risultato $R_5 = -128$ (saturato da -139).
- W_6, W_7: ...

Questo test verifica il comportamento del sistema con la minima lunghezza di sequenza W_i permessa da specifica. Particolare attenzione è posta sulla corretta gestione del padding, sull'intero processo di filtraggio (ordine 3, $n = 12$, normalizzazione per numeri negativi) e sulla saturazione dei risultati a seguito di valori intermedi estremi.

3.2.3. Test 3: Sequenza Corta (K=3), Filtro Ordine 5 e Padding Intensivo con Valori Standard

Input:

- Parametri in memoria: K=3, S=LSB='1' (Filtro Ordine 5).
- Coefficienti Ordine 5: $[123, 91, 13, -98, -91, 9, -41]$.

- Dati W_i: $W = [10, 20, -15]$.

Output Atteso (Valori R_i in memoria e stati finestra per i 3 output):

- Per $W_1 = 10$:
  - Finestra Scorrevole: $[0, 0, 0, 10, 20, -15, 0] \longrightarrow R_1 = -47$.
- Per $W_2 = 20$:
  - Finestra Scorrevole: $[0, 0, 10, 20, -15, 0, 0] \longrightarrow R_2 = -7$.
- Per $W_3 = -15$:
  - Finestra Scorrevole: $[0, 10, 20, -15, 0, 0, 0] \longrightarrow R_3 = 43$.

Questo test è stato sviluppato per stressare il sistema con una sequenza W_i (K=3) più corta della finestra del filtro (7 campioni), andando oltre i requisiti minimi della specifica di progetto (K $\geq$ 7) per valutarne la robustezza. Ciò richiede un padding intensivo. È importante notare che, nel caso limite in cui la lunghezza della sequenza di input `i_k` sia inferiore a 4, il numero di cicli dedicati all'inizializzazione della finestra scorrevole rimane comunque pari a 4. Durante questi cicli, se i campioni effettivi della sequenza W_i sono esauriti, verranno caricati degli zeri per completare il riempimento della finestra. Questa gestione del padding assicura che la logica di calcolo dell'indirizzo per la scrittura dei risultati rimanga invariata. Tuttavia, in tale scenario, il valore del `counter` al termine della lettura dei campioni W_i potrebbe non corrispondere all'indice dell'ultimo dato effettivamente letto dalla memoria (a causa degli zeri di padding inseriti), ma ciò non compromette la corretta generazione degli indirizzi di scrittura né il funzionamento complessivo del sistema.

3.2.4. Test 4: Gestione Esecuzioni Multiple e Reset del Sistema

Questo test è stato progettato per verificare la robustezza e la stabilità del componente in scenari operativi più complessi, che includono:

- **Esecuzioni consecutive:** Il componente deve essere in grado di processare una sequenza, terminare con `o_done`, e poi ripartire correttamente per una nuova elaborazione (con parametri e/o filtri potenzialmente diversi) semplicemente abbassando e rialzando il segnale `i_start`, senza la necessità di un reset esplicito `i_rst`. Questo verifica che lo stato interno venga correttamente reinizializzato dal segnale `component_reset` (legato a `i_start`='0').
- **Reset durante l'elaborazione:** Il segnale asincrono `i_rst` viene attivato a metà di un'elaborazione. Ci si aspetta che il componente interrompa l'operazione corrente, si resetti completamente allo stato `IDLE`, e sia pronto a ripartire da zero con una nuova sequenza quando `i_start` viene nuovamente asserito.

In sintesi, questo test valuta la capacità del sistema di operare in modo affidabile attraverso multiple sequenze di start/stop, reset improvvisi e diverse configurazioni di filtro, assicurando che ogni elaborazione sia indipendente e corretta.

4 | Conclusioni

Il progetto ha realizzato un modulo VHDL per filtraggio differenziale configurabile, conforme alle specifiche. La sintesi evidenzia l'efficienza del design (slack +6.237ns @50MHz e assenza di latch).

L'architettura modulare, con FSM centrale, è risultata robusta e le scelte implementative hanno permesso di soddisfare i requisiti funzionali e temporali.

Per futuri requisiti prestazionali più stringenti o elaborazioni più complesse, si potrebbe considerare l'introduzione di stadi di pipeline, ad esempio nelle unità MAC o di normalizzazione, per ridurre ulteriormente il numero di livelli logici per ciclo di clock.